

Findings — Central da Nação

Pesquisa, constraints e lacunas levantadas na **Phase 1 / Protocol 0** (2026-09-09), para consulta do System Pilot e dos agentes das fases L/A/S/T.

Fonte da verdade de regras: `gemini.md` + blueprint `BLAST_Central_da_Nacao_SYSTEM_PROMPT.md` v1.0. Este arquivo registra o que foi descoberto fora do blueprint.

1. Resumo executivo

- Não existe repositório público maduro e ativo **específico** de RSS do Flamengo; o caminho mais sólido é usar **RSSHub** (ou RSS nativo de ge/veículos da allowlist) para as fontes que não expõem feed.
- Não existe **wrapper público de api-futebol.com.br** (a API brasileira). Existem wrappers de **API-Football (api-sports.io)** e clientes do Brasileirão. Conclusão: o adapter de futebol será **próprio, atrás de interface**, com fixture mock no handshake (como o blueprint já prevê) — e vale a pena avaliar api-sports.io na Phase L por ter ecossistema maior.
- Para n8n, há coleções grandes de workflows e um pipeline de notícias RSS→LLM→armazenamento que espelha nosso fluxo (arquivado — usar só como referência).
- Todos os metadados de repositórios abaixo foram **verificados pela API pública do GitHub em 2026-09-09** (estrelas, licença, linguagem, data do último push, arquivado).

2. Pesquisa GitHub — Top 5 repositórios úteis

Critério: utilidade direta para os 3 eixos pedidos (RSS Flamengo, wrappers de API de futebol, n8n sports/news), licença clara, manutenção e risco de adaptação.

1. DIYgod/RSSHub — “Everything is RSSible” (eixo: RSS)

- **URL:** <https://github.com/DIYgod/RSSHub>
- **Linguagem:** TypeScript · **Licença:** AGPL-3.0 · **Estrelas:** ~46.1k · **Ativo:** push em 2026-09-09
- **O que é:** maior gerador de feeds RSS do mundo; transforma qualquer site em RSS por rotas de comunidade.
- **Por que é útil:** fontes da allowlist sem RSS nativo (ex.: flamengo.com.br/noticias, colunado-fla.com) podem virar feed via instância RSSHub. Serve de rota de ingestão determinística no `10_ingest_news`.
- **Riscos/adaptação:** AGPL-3.0 (atenção se for incorporar código — preferir usar via instância/API, sem copiar código para dentro das tools); instâncias públicas são instáveis → self-host só com decisão humana; validar na Phase L se cada domínio da allowlist tem rota RSSHub estável.

2. ezefranca/campeonato-brasileiro-api (eixo: dados de futebol BR)

- **URL:** <https://github.com/ezefranca/campeonato-brasileiro-api>
- **Linguagem:** JavaScript/Node · **Licença:** MIT · **Estrelas:** ~104 · **Ativo:** push em 2026-09-03
- **O que é:** API simples do Campeonato Brasileiro (Séries A–D) com CLI e suporte a MCP (Model Context Protocol).

- **Por que é útil:** referência de schema de rodadas/jogos/classificação em pt-BR e fonte secundária de conferência cruzada de jogos do Flamengo; o CLI/MCP pode servir de inspiração para as tools atômicas do MVP.
- **Riscos/adaptação:** não substitui a API-Futebol/API-Football (não dá placar ao vivo confiável); usar como apoio, não como fonte primária do match center.

3. h3ave/apisports_football (eixo: wrapper API-Football)

- **URL:** https://github.com/h3ave/apisports_football
- **Linguagem:** Python · **Licença:** MIT · **Estrelas:** 0 · **Último push:** 2024-10-29 (parado ~2 anos)
- **O que é:** wrapper assíncrono da API de futebol da API-SPORTS (api-sports.io).
- **Por que é útil:** o único wrapper Python dedicado à API-Football encontrado; base conceitual boa (endpoints, parâmetros, padrão async) para o adapter próprio previsto no blueprint.
- **Riscos/adaptação:** sem manutenção recente e sem testes abrangentes → **não copiar código cegamente**; usar como referência de chamadas e escrever nosso adapter atrás de interface com fixture mock.

4. Zie619/n8n-workflows (eixo: n8n)

- **URL:** <https://github.com/Zie619/n8n-workflows>
- **Licença:** MIT · **Estrelas:** ~56.6k · **Ativo:** push em 2026-06-24
- **O que é:** coleção massiva de workflows n8n prontos (RSS, webhooks, LLM, Sheets, Telegram etc.).
- **Por que é útil:** biblioteca de padrões para prototipar os triggers do dia (ingest 06:30, build 07:15, refresh 12:00/18:00) antes de codar as tools em Python.
- **Riscos/adaptação:** volume alto e qualidade variável → selecionar por padrão (RSS + AI + planilha) e adaptar, nunca importar inteiro; n8n é só apoio de prototipagem — a execução canônica do MVP são as tools Python definidas no blueprint.

5. jeremylongshore/news-pipeline-n8n (eixo: n8n sports/news)

- **URL:** <https://github.com/jeremylongshore/news-pipeline-n8n>
- **Linguagem:** workflows n8n (JS) · **Licença:** MIT · **Estrelas:** ~15 · **Status:** arquivado em 2026-03-25
- **O que é:** pipeline diário de notícias: coleta de múltiplos RSS → análise via LLM (sumarização, extração de entidades, scoring) → dados estruturados no Airtable.
- **Por que é útil:** é o **espelho conceitual do nosso fluxo** ingest_news → classify_and_dedupe → write_cards → destino cloud; bom para desenhar nós, campos e tratamento de erro.
- **Riscos/adaptação:** arquivado (sem manutenção) e configurado para tech news → usar **somente como referência de arquitetura**, nunca como base de código.

3. Menções e descartados (para não pesquisar de novo)

Repo	Veredito
pyanderson/python-brasileirao	MIT, Python, mas parado em 2021 e baseado em crawler da UOL → padrão simples só para estudo.
kariofreire/api-futebol-brasileiro	PHP/Laravel, sem licença e parado em 2022 → descartado (licença impede uso seguro).
ThiagoPax/flamengo-rss	0 estrelas, sem README, sem licença, inativo desde 2025-01 → descartado.
rishavganguly007/football-API-client	wrapper API-Football, 2 estrelas, 2023 → inferior ao #3.
enescingoz/awesome-n8n-templates	curadoria de 280+ templates (atualizado 2026-09) → útil como índice de templates; não é repo de workflows do nosso domínio.
BhaveshBhakta/N8N-Telegram-News-Agent	RSS → LLM → Telegram, 2025 → padrão de entrega de digest; fase 2 (WhatsApp/e-mail).
Topic <code>rss</code> , <code>sports-news</code> , <code>api-football</code> no GitHub	úteis para re-pesquisas futuras.

4. Constraints e decisões herdadas do blueprint (resumo operacional)

- **Timezone:** America/Sao_Paulo em tudo (horários de cron, ISO -03:00 nos fixtures).
- **Selos:** `oficial` só para domínio oficial allowlist (flamengo.com.br e redes oficiais); `rumor` quando fonte não oficial + texto com “negociando / perto / sondado / exclusividade” sem confirmação cruzada; ambíguo → `needs_review: true`.
- **Dedupe:** similaridade título+entidades > 0.82 → manter a mais antiga (+ a oficial se existir); `duplicate_of` preenchido.
- **Fontes/contas:** scraper ignora domínio fora da allowlist de `gemini.md`; novas fontes só por humano; máx. 20 contas no MVP; **nada de API do X no MVP** (timeline via embed/curation).
- **Futebol:** uma das duas APIs (api-futebol.com.br ou api-sports.io); key pode não existir → handshake com fixture mock atrás de interface.
- **Persistência default:** Google Sheets (`news_cards` , `matches` , `venues` [+ `sources` , `accounts` , `daily_payload`]) ou Supabase; `.tmp/` é bancada, verdade é cloud.
- **Publicação:** fila humana (✅/✏️/🗑️) entre 07:15 e 07:45; `AUTO_PUBLISH=false`; sem resposta → `status=draft`; ticks de jogo da API publicam sem aprovação.
- **Venues:** só `status=verified` + cidade do usuário/default; 5 cidades piloto; fora da lista não entra no Daily Payload do MVP.

- **Payload:** 4 seções obrigatórias; seção vazia = `items: []` + `status: "empty"` (chave nunca omitida).
- **Proibições:** sem paywall, sem escudo/marca oficial no produto, sem monetizar conteúdo oficial, sem vídeo FlaTV/Globo/ESPN em cortes (só UGC com crédito ou material próprio), sem se passar pelo clube (assinar “Central da Nação / veículo independente”).
- **Tom:** pt-BR, 2ª pessoa ou “a Nação”, direto, irônico permitido; nunca xenofobia/machismo/homofobia/ataque a menor; rival = provocar clube, não torcida como gente.

5. Lacunas para a Phase L (Link) — o que precisa existir antes dos pings

5.1 Credenciais pendentes (nenhuma presente hoje; nunca inventar credencial)

1. `API_FUTEBOL_KEY` — decidir **qual** provedor (`api-futebol.com.br` ou `api-sports.io`) e obter a key. Se não vier, Phase L segue com mock.
2. `LLM_API_KEY` — OpenAI **ou** Anthropic **ou** Gemini (escolher provedor; o nome da var é `LLM_API_KEY`).
3. Persistência — escolher **Google Sheets** (default; precisa `GOOGLE_SERVICE_ACCOUNT` + id da planilha) **ou** Supabase (`SUPABASE_URL` + `SUPABASE_KEY`).
4. `EDITOR_WEBHOOK_URL` — Slack ou Discord (escolher canal do editor).
5. Fase 2 (não bloqueiam o MVP): `RESEND_API_KEY` , `YOUTUBE_API_KEY` , `GOOGLE_MAPS_KEY` .

O checklist completo de variáveis está em `.env.example` (sem valores reais).

5.2 Riscos técnicos a validar no ping_rss

- Confirmar se cada domínio da allowlist expõe RSS/feed estável (`ge.globo` tem feed do time; `flamengo.com.br`, `colunadofla.com` etc. precisam de verificação prática).
- Se faltar feed: sinalizar ao humano (candidato: RSSHub self-host — decisão humana, não automática).

5.3 Regras de handshake (Phase L)

- Pings obrigatórios que travam o pipeline completo: **ping_sheets (1)**, **ping_webhook (2)**, **ping_rss (4)**. Football API pode ficar mockada (blueprint).
- Ping que falha → registrar aqui e em `progress.md` ; tools de lógica daquela integração não avançam; as demais seguem.

6. Log de atualizações deste arquivo

- **2026-09-09 (Protocol 0):** criado; pesquisa GitHub executada e metadados verificados via API do GitHub; top 5 definido; constraints consolidadas; lacunas de credenciais listadas.